Lead Scraper Suite
Build Modules — treg Action Plan

Every module required to go from an empty folder to two working, independent lead scrapers (LinkedIn and Google) that output lead CSVs with emails, phone numbers and every public company detail. For each module: **what to do** and **exactly how to do it**, based on the Lead Scraper PRD v1.0 (Oct 29, 2026) and the treg registry (github.com/superdesigndev/treg). **Updated:** Google covers whole states (state -> cities -> ZIP codes) and scrapes each website in full; LinkedIn finds people through a people-database search, with Google X-ray as fallback (e.g. HR in Texas), and gets their work emails. **M1 results (Oct 29) are folded in.**

Module	Scope	Priority	Current State
PHASE 1: SHARED CORE			
M0	Project Setup & treg Account	P0	Done (Oct 29): venv, treg CLI, token in .env, git
M1	Endpoint Discovery (treg Catalog)	P0	Done (Oct 29): endpoints.json + docs/endpoints.md
M2	treg Client & Cost Control	P0	Done (Oct 29): common.py Treg + offline checks
M3	Lead Model, CSV Output & Resume	P0	Done (Oct 29): LeadWriter, columns, Excel-checked
M4	Website Deep Scrape (Full Details)	P0	Done (Oct 29): 18/18 reachable sites read, email 10/18
M5	Paid Enrichment (Email Finder, Verify, Phone)	P0/P1	Done (Oct 30): Hunter 3/8 found, coverage 13/18
PHASE 2: GOOGLE SCRAPER			
M6	Google Maps Search	P0	Does not exist
M7	Google Scraper CLI & Run Summary	P0	Does not exist
M8	Area Coverage (State -> Cities -> ZIP Codes)	P0	Does not exist
PHASE 3: LINKEDIN SCRAPER			
M9	People Discovery (Search + X-Ray Fallback)	P0	Does not exist (source chosen in M1)
M10	Profile Check & Fallback Lookup	P0	Does not exist
M11	Work Email & Phone Finder	P0	Does not exist (tiers measured in M1)
M12	LinkedIn Scraper CLI & Company Mode	P0/P1	Does not exist
CROSS-CUTTING: FINISH BEFORE REAL USE			
M13	Errors, Logging & Run Summary	P1	No error handling exists yet
M14	Compliance & Suppression List	P1	Not designed yet
M15	Testing & README	P1	No tests exist
M16	Graphify Architecture Update	Final	Do last, after implementation

Priority: P0 = needed for the core loop (search -> details -> enrich -> CSV) · P1 = needed before real client use · Final = after implementation. "Current State" assumes an empty project folder. File names are the proposed structure from the PRD (linkedin_scraper.py, google_scraper.py, common.py, endpoints.json); rename them if you prefer.

Recommended working rule: implement one module at a time, in the order above. Test every module with **--dry-run and tiny limits (5-10 leads)** first, check the cost in the run log, then run it for real and review the diff before moving on. Do not attempt both scrapers in one step. Phase 2 (Google) comes before Phase 3 (LinkedIn) because Maps listings already include phone and website, so the shared enrichment gets tested on the easier source first. Module M16

(Graphify) is always last.

How to use this plan

- **One module = one work session.** Ask Claude: *"Implement M2 from Lead-Scraper-Build-Modules.pdf"*. Each module lists its goal, steps, files and a **Done when** check, so it can be built and verified on its own.
- **Check "Done when" before moving on.** If it fails, fix it inside the same module. Later modules assume earlier ones work.
- **M1 decides everything after it.** Do not start M6 or M9 until M1 has real endpoint IDs and sample responses; if a LinkedIn or Google capability is missing, change the plan there, not halfway through a scraper.
- **Update the Current State column** as modules finish (Not started -> In progress -> Done) so this file stays the project's progress tracker.
- **Need only one scraper?** Build Phase 1 + that scraper's phase + M13-M15. Google-only skips M9-M12; LinkedIn-only skips M6-M8, except the US city list from M8 step 1, which M9 needs.

M0 — Project Setup & treg Account

PHASE 1 · P0

Goal: *Create the Python project, the treg account and the token every later module uses. Nothing here scrapes yet; it makes sure the first real call in M1 works on this Windows machine.*

Used by: Both scrapers

What to do

- Create the Python project and a virtual environment.
- Install the treg CLI and sign in.
- Create a team token for the scrapers (not your personal login).
- Store the token outside the code, in a git-ignored .env file.
- Put the folder under git from day one.

How to do it

1. In `C:\Users\QS\Desktop\linkedin_scraper` run `python -m venv .venv` then `.venv\Scripts\activate`. Python 3.11+ (already installed).
2. Install the treg CLI with `pip install tools-registry` (on macOS/Linux the README's `curl -fsSL https://treg.to/install.sh | sh` also works). Run `treg login` and sign in with GitHub or the emailed code.
3. Run `treg` with no arguments: it prints the account, active team and balance. New teams start with $1.00 free credit, enough for M1 testing.
4. Create an agent token for scripts with `treg org agent-new`. This is the `TREG_TOKEN` the scrapers send; it can be revoked without touching your login.
5. Create `.env` with `TREG_TOKEN=...` and `.env.example` with the key name only. Add `.env`, `.venv/`, `*.csv` and `runs/` to `.gitignore` so tokens and lead data never reach git.
6. Run `git init` and make the first commit (PRD + setup files only).

Key files: `requirements.txt (new)` · `.env.example (new)` · `.gitignore (new)`

New dependencies: `treg CLI (tools-registry)` · `httpx` · `phonenumbers`

Done when: `treg balance` prints a balance and `.env` holds a working token.

M1 — Endpoint Discovery (treg Catalog)

PHASE 1 · P0

Goal: *Find the current best treg endpoint for every data capability, record its ID, price and real hit rate, and save one sample response each. This is the most important module: if a capability is missing, we find out here, before writing scraper code.*

Used by: Both scrapers

What to do

- Search the catalog for each capability in the table below.
- Check price and parameters before spending.
- Make one small test call per endpoint and save the response.
- Write the chosen IDs into endpoints.json.
- Measure email hit rate on 20 sample companies.

How to do it

1. For each capability run `treg catalog search "<job in plain words>"`, e.g. `"linkedin company search"`, `"linkedin company profile"`, `"linkedin profile"`, `"google maps places search"`, `"google place details"`, `"google search results"`, `"extract web page"`, `"email by domain"`. Search by the job, not the vendor name.
2. Prefer routed `treg.*` endpoints (treg picks the provider). Known ones: `treg.people.email.find`, `treg.people.email.verify`, `treg.people.phone.find`, `treg.people.search`, plus `hunter.people.email.find`.
3. Run `treg catalog get <id>` on each candidate to read price, parameters and notes.
4. Test with one call: `treg --json call <id> --query K=V > samples/<capability>.json`. Check the charged cost and `_treg.served_by` in the response.
5. Write `endpoints.json`: one entry per capability with `id`, `price_usd`, `params` and a note. Scrapers read IDs only from this file.
6. Run the email finder on 20 real company domains and note how many return an email. This number replaces the guessed 40-60% target in M15. Do the same for work emails on 20 LinkedIn profile URLs.
7. Measure two caps M8 and M9 depend on: the most results one Maps search returns (paging to the end), and how many results one Google-results query returns.
8. If a capability is missing: run `treg catalog request` to ask for it, or add your own provider key to treg with `treg tool add` (e.g. a Google Places API key) so it is called at no treg charge.

Capability	Used by	Search phrase / known ID
Google Maps / Places search	M6, M8	"google maps places search"
Google place details	-	not needed: Maps rows are complete (M1)
Google web results (SERP)	M9, M10	"google search results"
Web page extract (JS sites)	M4	"extract web page"
People search	M9	icypeas.people.search
LinkedIn person profile	M10	treg.people.enrich
LinkedIn company profile	M10, M12	"linkedin company profile"
LinkedIn company search	M12	"linkedin company search"
Email by domain (companies)	M5	hunter.companies.emails (opt-in)
Work email by person	M11	treg.people.email.find
Email verify / phone find	M5, M11	treg.people.email.verify, treg.people.phone.find

Key files: `endpoints.json (new) · samples/*.json (new, git-ignored) · docs/endpoints.md (new, notes)`

New dependencies: `none (treg CLI from M0)`

Done when: Every capability has an ID, price and saved sample, or a written decision on how to replace it.

M2 — treg Client & Cost Control

PHASE 1 · P0

Goal: *One function that is the only way any scraper talks to treg. It adds the token, retries, logs every call, and stops the run before it spends more than the user allowed.*

Used by: Every module that calls treg (M4-M12)

What to do

- `call(capability, params)` that reads the endpoint ID from endpoints.json.
- Token, retries and timeouts in one place.
- Running cost total from the X-Treg-Cost-Micro header.
- A hard budget (--max-cost) and a per-call cap.
- Dry-run cost estimate using catalog prices.

How to do it

1. In `common.py` create one `httpx.Client` with base URL `https://treg.to`, header `X-Treg-Token` from the environment, and a 180 s timeout (a routed email lookup took 77 s in M1). Agent tokens get 403 on `GET /orgs`; use `GET /auth/me` for the org id.
2. `call()` sends `GET /call/<id>?params` (or POST with JSON when the endpoint needs a body) and returns the parsed JSON. The provider's response comes back unchanged.
3. Read `X-Treg-Cost-Micro` from every response and add it to a module-level `spent_micro` counter. Send `X-Treg-Route-Max-Cost` on every call, from `max_cost_usd` in endpoints.json. treg's default is $1 per call and routed endpoints bill misses, so this header is never optional.
4. Before each call, if `spent + price > max_cost` raise `BudgetReached`; the scraper catches it, flushes the CSV and prints the summary.
5. Status handling: 429 and 5xx retry 3 times with backoff (1 s, 4 s, 10 s); 402 = out of balance, stop the run and print the top-up URL from the response; 503 = provider empty, not charged, skip the lead; 502 = response too large, not charged, skip.
6. Append one JSON line per call to `runs/<timestamp>.jsonl`: capability, endpoint, status, cost, served_by, duration. Never log the token.
7. `estimate(plan)` multiplies the prices in endpoints.json by expected call counts for `--dry-run`, using the formula in M7.

Key files: `common.py (new: call, BudgetReached, estimate) · endpoints.json`

New dependencies: `httpx`

Done when: A test call through call() returns JSON, cost is logged, and --max-cost 0 stops before any spend.

M3 — Lead Model, CSV Output & Resume

PHASE 1 · P0

Goal: *Fix the CSV columns once and write leads safely: row by row, de-duplicated, resumable and Excel-friendly. Both scrapers share the first 12 columns so their files can be stacked later.*

Used by: Both scrapers

What to do

- Define the shared 12 columns and each scraper's extra columns.
- Write each lead as soon as it is finished.
- Skip leads already in the file (resume).
- Normalize values: phones, domains, multi-value fields.

How to do it

1. Shared columns, in order: `company_name, website, domain, email, all_emails, email_source, phone, all_phones, address, city, country, source_url`.
2. LinkedIn people extras (people mode): `full_name, first_name, last_name, job_title, headline, person_location, person_linkedin_url, company_linkedin_url, industry, company_size, email_status, email_confidence, search_query, scraped_at, cost_usd`.
3. LinkedIn company extras (company mode): `linkedin_id, tagline, description, industry, company_size, employees_on_linkedin, headquarters, all_locations, founded_year, company_type, specialties, followers, logo_url, funding_info, socials, email_status, scraped_at, cost_usd`.
4. Google extras: `place_id, category, all_categories, state, postal_code, latitude, longitude, rating, review_count, opening_hours, google_maps_url, facebook, instagram, linkedin, x, youtube, tiktok, site_description, services, team_members, contact_form_url, site_address, search_query, search_location, email_status, scraped_at, cost_usd`.
5. Use stdlib `csv.DictWriter` with encoding `utf-8-sig` (Excel opens it correctly), `extrasaction="ignore"`, and `flush()` after every row so a crash loses at most one lead.
6. Resume: on start, if the output file exists, read its dedupe key column (`place_id` or `person_linkedin_url` or `source_url`) into a set and skip those leads before any paid call.
7. Multi-value fields are joined with `" | "`. Empty stays empty (never "N/A"). Domains are lower-case without `www.`. Phones go to international format (`+1 512-474-1234`) with `phonenumbers`: Excel shows bare E.164 as 1.51E+10 (checked in M3).

Key files: `common.py (LeadWriter, normalize_phone, domain_of) · columns defined at top of each scraper file`

New dependencies: `phonenumbers`

Done when: Writing, killing and re-running produces no duplicate rows; the file opens cleanly in Excel.

M4 — Website Deep Scrape (Full Details)

PHASE 1 · P0

Goal: *Visit every business website and collect everything useful on it: contact details (emails, phones, socials) plus what the company is and does (description, services, team, address, hours, contact form). This is free and runs before any paid lookup; LinkedIn and Google listings almost never show an email.*

Used by: Google scraper (every place with a website) · LinkedIn company mode

What to do

- Crawl the homepage and the key internal pages (contact, about, services, team).
- Read structured data (schema.org JSON-LD) first, then the page text.
- Extract emails, phones and social links, and filter out junk.
- Extract company details: description, services, team members, address, hours, contact form.
- Fall back to treg's page-extract only for JavaScript-only sites.

How to do it

1. Fetch the homepage with httpx (10 s timeout, follow redirects, a normal browser User-Agent). From its links pick up to 8 internal pages whose URL or link text matches contact, about, team, staff, services, what-we-do, locations or impressum; if none are linked, try `/contact` and `/about`.
2. Respect robots.txt with stdlib `urllib.robotparser`; skip disallowed paths.
3. Structured data first: parse every `<script type="application/ld+json">` block (stdlib json) for Organization / LocalBusiness: name, telephone, email, address, openingHours and `sameAs` (social links). When present this is the most reliable source.
4. Emails: `mailto:` links and regex matches in the text; decode Cloudflare-protected addresses (`data-cfemail`, a simple XOR) and "name [at] domain [dot] com" forms. Drop junk: image names (logo@2x.png), placeholders (example.com, sentry, wixpress) and other-domain emails unless they are the only ones. Rank: same-domain named > same-domain generic (info@, contact@) > other.
5. Phones: `tel:` links first, then `phonenumbers.PhoneNumberMatcher` for the lead's country. Socials: facebook.com, instagram.com, linkedin.com/company, x.com / twitter.com, youtube.com, tiktok.com.
6. Company details: `site_description` from the meta / og:description, else the first paragraph of the About page (max 500 chars); `services` from h2/h3 headings on the services page (max 15); `team_members` as "Name - Title" pairs from the team page where they can be detected; `contact_form_url` = first page with a form that has an email field; `site_address` from JSON-LD or the contact page.
7. If the homepage loads but has almost no text (a JavaScript app) and nothing was found, render it once through the page-extract capability via call() (paid, small). Sites that answer with a bot check (403 / challenge page) are skipped and reported, never worked around.
8. Run sites in a `ThreadPoolExecutor(max_workers=8)`; set `email_source="website"`.

Key files: `common.py (crawl_site, extract_contacts, extract_details)`

New dependencies: `none new (httpx, phonenumbers, stdlib re / json / html.parser / urllib.robotparser)`

Done when: On 20 known websites, contacts, description and services that are visible on the site are found, with no junk emails.

M5 — Paid Enrichment (Email Finder, Verify, Phone)

PHASE 1 · P0/P1

Goal: *Fill the email gap that M4 leaves, using paid finders only when the free crawl found nothing, and optionally verify emails before they go into the CSV.*

Used by: Both scrapers

What to do

- Opt-in paid domain search (`--paid-emails`) when M4 found no email (P1).
- Optional email verification with --verify (P1).
- Record the source of every email.

How to do it

1. With `--paid-emails` (off by default), if `enrich_website` returned no email and the lead has a domain, call `hunter.companies.emails` with `domain=acme.com&limit=10` (one credit covers up to 10 emails). Set `email_source` to hunter.
2. It reserves about $0.0245 per domain (free when nothing is found), which is why it is opt-in; its cap in endpoints.json is `0.03`.
3. With `--verify`, call `treg.people.email.verify` on the best email only and write `email_status` = valid / risky / invalid. Invalid emails move out of `email` but stay in `all_emails`.
4. Never look up the same domain twice in one run: keep a dict cache domain -> result.
5. `--no-enrich` skips M4 and M5 entirely (cheapest run, source data only).

Key files: `common.py (enrich_paid)`

New dependencies: `none new`

Done when: Leads without a website email get one from the finder where available; cost per lead is logged.

M6 — Google Maps Search

PHASE 2 · P0

Goal: *Turn one search cell, a business type plus one city or ZIP code ("dentists" in "Austin, TX"), into unique businesses with every field Google shows: website, phone, address, rating, reviews, hours, status. M8 runs many cells to cover a whole state.*

Used by: Google scraper

What to do

- Search Google Maps / Places with paging.
- Detect when a cell hits the result cap, so M8 can split it.
- No place-details call: search rows are already complete (M1: 100/100).
- Map provider JSON to the Google CSV columns.
- Filters: minimum rating, minimum reviews, website required.

How to do it

1. Call `crawl4ai.google.serp.maps` with `query`, `location` = the cell and `limit` 100: one call returns up to 100 places for $0.00015.
2. Count the results. If the count reaches the provider's cap (100, measured in M1), mark the cell saturated: there are probably more businesses than one search can return.
3. De-duplicate by `place_id` across the whole run; fallback key = normalized phone + domain.
4. No details call: in M1 every row already had website, phone, address, hours, rating and place_id.
5. Write one function `map_place(raw) -> dict`; a vendor change only touches it and endpoints.json.

6. Apply `--min-rating`, `--min-reviews`, and with `--require-website` keep only businesses that have a website. Filters run before the website scrape, so filtered leads cost nothing more.
7. Set `source_url` = Google Maps URL, `search_query` and `search_location` on every row.

Key files: `google_scraper.py (search_cell, map_place)`

New dependencies: `none new`

Done when: "dentists" in "Austin, TX" with --limit 50 --no-enrich gives 50 unique rows with phone for 90%+.

M7 — Google Scraper CLI & Run Summary

PHASE 2 · P0

Goal: *One command: area plan -> Maps search -> details -> website deep scrape -> CSV, with dry-run, budget and resume.*

Used by: Google scraper

What to do

- argparse command line.
- Pipeline wiring M2-M6 and M8.
- Dry-run estimate and end-of-run summary with coverage report.

How to do it

1. Flags: `--query` (required), `--location` (city, "City, ST", state name or code), `--split auto|none|cities|zips` (auto = cities when the location is a state), `--min-population`, `--locations-file`, `--limit`, `--out` (default leads_google.csv), `--max-cost` (default 5), `--dry-run`, `--resume`, `--no-enrich`, `--verify`, `--min-rating`, `--min-reviews`, `--require-website`, `--suppress`.
2. Examples: `python google_scraper.py --query "dentists" --location "Austin, TX" --limit 300` and `python google_scraper.py --query "IT companies" --location "Texas" --require-website --max-cost 25`.
3. Loop: for each cell from M8 -> M6 search -> for each new place: M4 website deep scrape -> M5 if no email -> write row. Print progress every 10 leads: count, emails found, spend so far.
4. Dry-run: number of cells x pages x search price + expected leads x (details share + extract share + email-finder share (+ verify with `--verify`)). Print the total and exit with zero spend.
5. Ctrl+C and `BudgetReached` both end cleanly: file flushed, summary printed.
6. Summary: leads written, % with website / email / phone, cells searched / saturated / re-split, total cost, cost per lead, errors, path of the run log.

Key files: `google_scraper.py (main)`

New dependencies: `none new`

Done when: Acceptance tests G-A1 to G-A4 in M15 pass.

M8 — Area Coverage (State -> Cities -> ZIP Codes)

PHASE 2 · P0

Goal: *Get as close as possible to every matching business in a large area. One Maps search returns at most 100 places (M1), so a state is searched city by city, and any city that hits the cap is searched again ZIP code by ZIP code.*

Used by: Google scraper (and M9 reuses the city list)

What to do

- A built-in list of US cities (with population) and ZIP codes per state.
- Split a state into cities; re-split saturated cities into ZIP codes.
- One dedupe set and one budget across all searches.
- Custom area lists for any country via a file.
- A coverage report.

How to do it

1. Build two small data files once from the free GeoNames exports (CC BY 4.0): `data/us_cities.csv` (city, state, population, lat, lon) from cities1000, and `data/us_zips.csv` (zip, city, state, lat, lon) from the US postal-code file. Commit both; no runtime download.
2. `--location "Texas"` (or `TX`) -> every Texas city with population at or above `--min-population` (default 5,000), largest first. `--location "Austin, TX"` -> one cell.
3. Run M6 per city. When a city comes back saturated, run it again once per ZIP code of that city. ZIP cells are never split further.
4. All cells share M6's `place_id` set, so a business found from two neighbouring cities is scraped and paid for once.
5. Results bleed across borders: drop places whose address state differs from the requested state.
6. `--locations-file areas.txt`: one location per line (any country), searched as given, no splitting.
7. Coverage report in the summary and run log: cells searched, saturated, re-split, unique places. Dry-run uses the number of cells to estimate search cost.

Key files: `google_scraper.py (plan_cells, run_cells) · data/us_cities.csv (new) · data/us_zips.csv (new)`

New dependencies: `none new`

Done when: "dentists" in "Texas" returns far more unique places than one statewide search, with no duplicates and a coverage report.

M9 — People Discovery (Search + X-Ray Fallback)

PHASE 3 · P0

Goal: *Turn "HR people at tech companies in Texas" into people with LinkedIn URL, current title, company and company website, using a people-database search through treg (no LinkedIn login). M1 measured it at 7,532 matches for this example, 20/20 in Texas. Google X-ray search is the fallback.*

Used by: LinkedIn scraper (people mode)

What to do

- Expand the role into job-title variants.
- Search `icypeas.people.search` with title, profile location and industry filters, paging to the limit.
- De-duplicate by profile.
- Fallback `--source xray`: Google X-ray search for LinkedIn profile URLs.

How to do it

1. Role synonyms live in a small dict in linkedin_scraper.py, e.g. `HR` -> HR Manager, HR Director, Human Resources, Talent Acquisition, Recruiter, People Operations. `--titles` overrides the list.

2. Query: `currentJobTitle` include the variants; `profileLocation` include `"Texas, United States"` (M1: 20/20 in-state; the plain `location` filter gave 13/20); `currentCompany.industry` include LinkedIn industry names (tech -> Software Development, IT Services and IT Consulting, Technology, Information and Internet); `--size` maps to `currentCompany.headcount`.
3. Page with `pagination.size` = min(100, rows still needed) and pass back `pagination.token` until --limit or no token. Size is the price dial: $0.00038 per row returned.
4. De-duplicate by the profile slug of `profileUrl`; store `search_query` on each person.
5. X-ray fallback: `treg.google.serp.organic` with `site:linkedin.com/in ("HR Manager" OR "Recruiter") "Austin" "software"`, limit 100 ($0.0015, 76 profiles in M1), 3-4 titles per query, state expanded to cities from `data/us_cities.csv`. Parse `Name - Title - Company | LinkedIn` from result titles; these rows go through M10's lookup.

Key files: `linkedin_scraper.py (title_variants, people_search, xray_search, normalize_profile_url)`

New dependencies: `none new`

Done when: "HR" + tech + Texas returns the requested number of unique people, with company website for 90%+.

M10 — Profile Check & Fallback Lookup

PHASE 3 · P0

Goal: *Make sure every person really matches (current title, requested state) and has a company domain for the email finder. People-search rows already carry company name, website, size and industry, so paid lookups are only needed for X-ray rows and missing domains.*

Used by: LinkedIn scraper

What to do

- Local hard filters with logged reasons.
- Company domain from the row's company website.
- Paid lookups only for X-ray rows and missing domains.
- Map to the people CSV columns.

How to do it

1. Filters: the current title matches a title variant; the address is in the requested state (state name, code, or a city / metro from `data/us_cities.csv`, e.g. "Dallas-Fort Worth"). Each rejection is logged with its reason.
2. Domain = host of `lastCompanyWebsite` without www. Missing: one `treg.companies.search` call by company name (~$0.0002), else leave empty and skip tier-2 email lookups.
3. X-ray rows: `treg.people.enrich` with the profile URL ($0.000495) for name, title, company and location, then the same filters.
4. Map icypeas fields: `firstname, lastname, headline, lastJobTitle, address, profileUrl, lastCompanyName, lastCompanyUrl, lastCompanyWebsite, lastCompanySize, lastCompanyIndustry` to the people columns from M3.

Key files: `linkedin_scraper.py (check_person, company_domain, map_person)`

New dependencies: `none new`

Done when: On 20 sample people the domain is filled for 90%+, and out-of-state or non-matching people are dropped with a reason.

M11 — Work Email & Phone Finder

PHASE 3 · P0

Goal: *Find each qualified person's work email, and optionally phone. LinkedIn does not show emails publicly, so they come from email-finder providers through treg: a cheap tier first, a capped fallback for misses.*

Used by: LinkedIn scraper (people mode)

What to do

- Tier 1: cheap LinkedIn-URL finder (misses free).
- Tier 2: routed finder for tier-1 misses only, capped.
- Optional verify and phone.
- Never pay twice for the same person.

How to do it

1. Tier 1: `blitzapi.people.email.find` with `person_linkedin_url`: $0.000495 per hit, misses free; M1 hit rate 15/30 (50%).
2. Tier 2 for misses: `treg.people.email.find` with linkedin_url, first/last name and domain; headers `X-Treg-Route-Max-Cost: 0.011` and `X-Treg-Route-Exclude: blitzapi,enrow`; 180 s timeout. About $0.006 per attempt and 1 in 4 hits in M1. `--no-fallback` skips it.
3. Never send `X-Treg-Route-Verify` on the finder: in M1 it tried 5 providers, ran past 120 s and cost $0.018. With `--verify`, call `treg.people.email.verify` separately (up to $0.00124).
4. Write `email`, `email_status`, `email_confidence`, and `email_source` = the provider that served it. No result: leave it empty; never write unverified guesses such as first.last@domain.
5. With `--phone` call `treg.people.phone.find` (cap $0.005; no provider bills misses).
6. The resume file (M3) is keyed by profile slug, so re-runs never pay for the same person again.

Key files: `linkedin_scraper.py (find_contact)`

New dependencies: `none new`

Done when: On 20 qualified people about half or more get a work email (M1 rate), and nobody is looked up twice.

M12 — LinkedIn Scraper CLI & Company Mode

PHASE 3 · P0/P1

Goal: *One command with two modes: people (default, P0) for contacts like HR managers, and company (P1) for company-level leads.*

Used by: LinkedIn scraper

What to do

- People-mode command line (P0).
- Company mode: company search or URL list -> company profile -> website deep scrape (P1).
- Same common flags as the Google scraper.

How to do it

1. People: `python linkedin_scraper.py --role "HR" --industry tech --location "Texas" --limit 300 --max-cost 20 --out hr_texas.csv`. Flags: `--role`, `--titles`, `--industry`, `--location`, `--fast`, `--phone`, `--limit`.
2. Company: `python linkedin_scraper.py --mode company --keyword "SaaS" --location "Texas" --limit 200` or `--input company_urls.txt`. Discovery via `treg.companies.search` (50 companies with website, phone and LinkedIn URL for ~$0.0002 in M1); URL lists via `treg.linkedin.company.profile`; then M4 / M5 for company emails.

3. Common flags: `--max-cost --dry-run --resume --no-enrich --verify --suppress --out`.
4. People pipeline: M9 queries -> M10 profile + filters -> M11 email -> write row; progress every 10.
5. Dry-run: queries x pages x search price + expected profiles x (profile price + company price shared per company + email price).
6. Never import google_scraper.py, and vice versa: shared code lives only in common.py.

Key files: `linkedin_scraper.py (main)`

New dependencies: `none new`

Done when: Acceptance tests L-A1 to L-A4 in M15 pass.

M13 — Errors, Logging & Run Summary

CROSS-CUTTING · P1

Goal: *Make every failure visible and safe: a bad website or a provider outage loses one lead, never the run.*

Used by: Both scrapers

What to do

- One clear message per failure type.
- Per-lead error isolation.
- Run log you can read after the fact.

How to do it

1. Wrap each lead in try/except: log the error with the lead key, write the row with what was collected (empty email/phone), continue.
2. Website timeouts, SSL errors and 403s are expected: count them, don't print a stack trace.
3. Missing `TREG_TOKEN` or an unknown capability in endpoints.json fails at start-up with one line telling the user what to fix.
4. The JSONL run log (M2) plus an `errors` count in the summary is all the logging needed; no logging framework.

Key files: `common.py · both scrapers`

New dependencies: `none new`

Done when: Pulling the network cable mid-run ends with a saved CSV and a readable summary.

M14 — Compliance & Suppression List

CROSS-CUTTING · P1

Goal: *Keep the lead lists usable legally: no opted-out contacts, sources recorded, data kept local.*

Used by: Both scrapers

What to do

- Suppression list applied before writing.
- Source recorded for every email and phone.
- Owner reads provider terms and outreach rules once.

How to do it

1. `--suppress suppress.txt`: lines are emails or domains; matching emails are removed, matching domains drop the whole lead. Applied before the row is written.
2. `email_source` and `source_url` are always filled, so any contact can be traced to where it came from.

3. Never log in to LinkedIn or automate a LinkedIn account (it gets restricted fast and breaks LinkedIn's terms); profile data comes from providers through treg. Read each provider's terms shown in `treg catalog get`.
4. Outreach to EU/UK contacts needs a documented lawful basis (GDPR); US email outreach must follow CAN-SPAM (identification + unsubscribe). Named personal emails are personal data; generic info@ addresses are lower risk.
5. CSV files stay on this machine and are git-ignored (M0).

Key files: `common.py (load_suppression) · suppress.txt (user-maintained)`

New dependencies: `none new`

Done when: A domain in suppress.txt never appears in any output CSV.

M15 — Testing & README

CROSS-CUTTING · P1

Goal: *One small test file for the logic that can silently go wrong, real acceptance runs for each scraper, and a README so the tool can be used without this document.*

Used by: Both scrapers

What to do

- Unit checks for parsing, query building and CSV logic.
- Acceptance runs against real treg endpoints.
- README with setup and examples.

How to do it

1. `test_common.py` (plain asserts): email extraction incl. Cloudflare and [at] forms, junk filtering, JSON-LD parsing, phone format, profile and company URL normalization, LinkedIn result-title parsing, X-ray query length, state -> cities planning, resume skips existing keys, budget stop.
2. G-A1: "dentists" / "Austin, TX" / --limit 50 gives 50 unique rows; phone 90%+, website 80%+, email at or above the rate measured in M1, description filled for most sites. G-A2: --max-cost 0.10 stops at or below $0.10. G-A3: kill mid-run, --resume gives no duplicates and no repeat paid calls.
3. G-A4: "dentists" / "Texas" / --limit 1000 splits into cities, re-splits saturated cities by ZIP, no duplicates, coverage report printed.
4. L-A1: --role HR --industry tech --location Texas --limit 50 gives 50 unique people with name, title, company and profile URL, all in Texas with a matching title; work email at or above the M1 rate. L-A2: --mode company --input with 20 URLs returns exactly those 20. L-A3: --dry-run leaves the treg balance unchanged. L-A4: --resume never repeats an email lookup.
5. X-A1: every CSV opens in Excel with correct characters, and the first 12 columns match in name and order.
6. `README.md`: setup (M0), commands with examples, all flags, columns, cost notes, compliance notes.

Key files: `test_common.py (new) · README.md (new)`

New dependencies: `pytest (dev, optional)`

Done when: All checks pass and the owner signs off on one real CSV from each scraper.

M16 — Graphify Architecture Update

FINAL

Goal: *Update the project's knowledge graph so it shows the real, final architecture, only after the scrapers are built and working.*

Used by: Project documentation

What to do

- Represent the built scrapers and shared core in Graphify.
- Do this last.

How to do it

1. Once M0-M15 (or the P0 subset) work, run the /graphify skill on the project folder.
2. Make sure it shows: the two scraper entry points, common.py (treg client, enrichment, CSV writer), endpoints.json and the treg capabilities behind it, and the CSV outputs.
3. Do this after implementation, so the graph documents what was actually built rather than this plan.

Key files: `graphify-out/ (generated)`

New dependencies: `none`

Done when: The graph matches the code.

Lead Scraper Suite · Build Modules — treg Action Plan · Build specification derived from the Lead Scraper PRD v1.0 and the treg registry documentation.